

Agent程序

祝恩
计算机学院
国防科技大学



学习内容

1. 4种类型的Agent程序结构
2. 通过学习获得Agent程序的能力
3. Agent程序的状态如何表示

1 AGENT程序的结构



Agent的结构

- How does the inside of the agent work?
 - Agent = architecture + program
- All agents have the same skeleton
 - Input = current percepts,
 - Output = action
 - Program = manipulates input to produce output

Agent程序的四种基本结构

- Four basic types in order of increasing generality
 1. Simple reflex agents
 2. Model-based reflex agents
 3. Goal-based agents
 4. Utility-based agents
- All these can be turned into learning agents

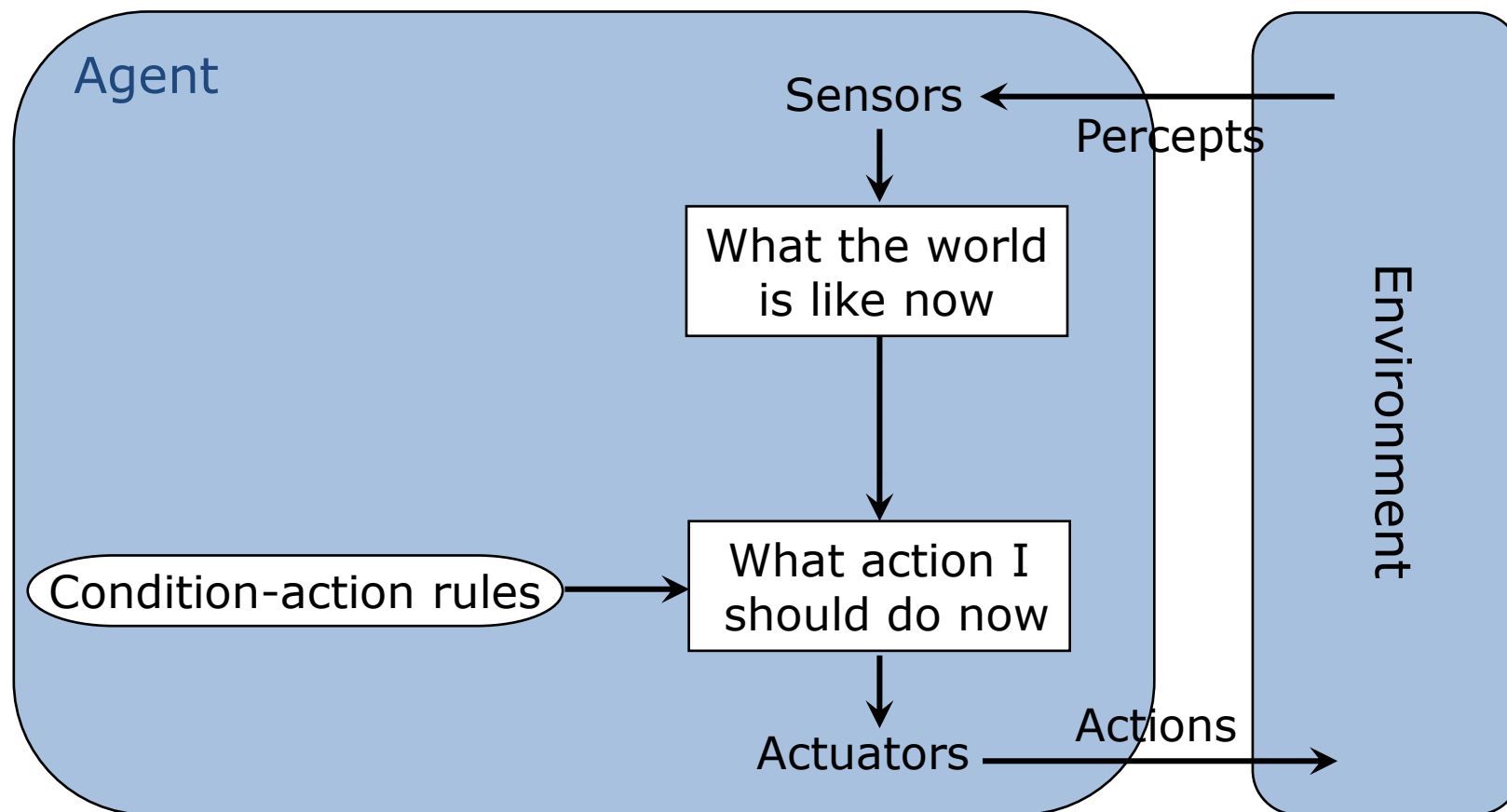


(1)简单反射AGENT

简单反射Agent Simple Reflex Agent

- 最简单的Agent种类是简单反射Agent。这类Agent基于当前的感知选择行动，不关注感知历史。

简单反射Agent



简单反射Agent

```
function REFLEX-VACUUM-AGENTS([location, status]) return an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

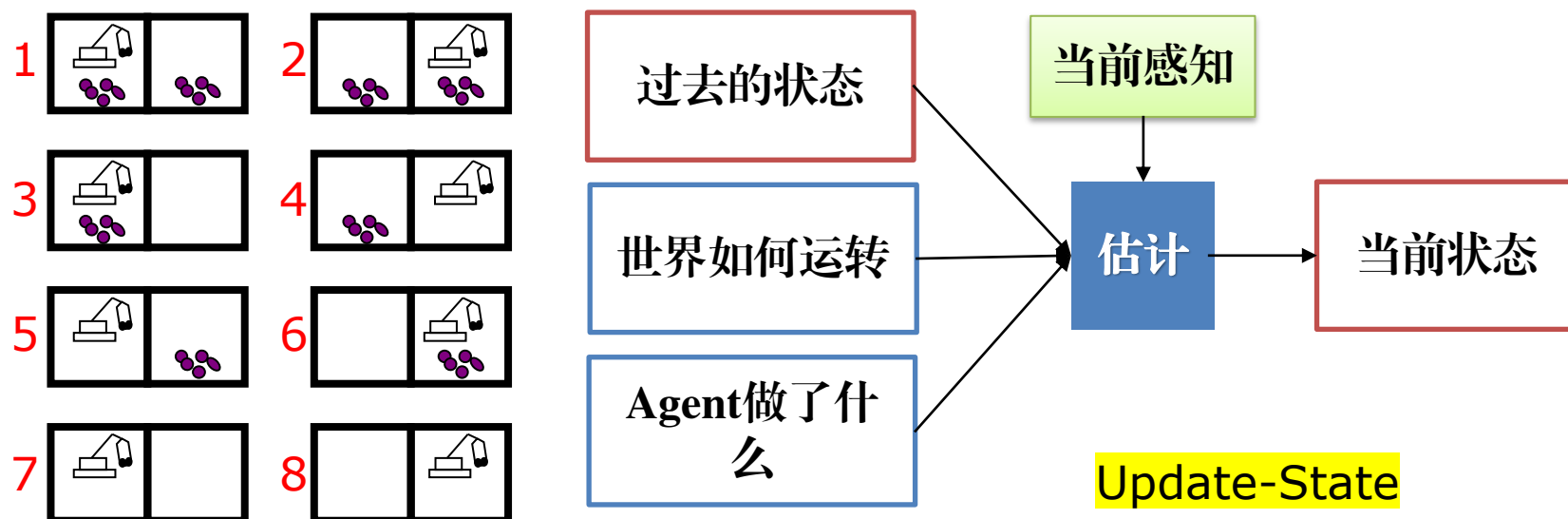
```
function SIMPLE-REFLEX-AGENT(percept) returns an action  
  static: rules, a set of condition-action rules  
  
  state <- INTERPRET-INPUT(percept);  
  rule <- RULE-MATCH(state, rules);  
  action <- rule.ACTION;  
  return action
```



(2)基于模型的反射AGENT

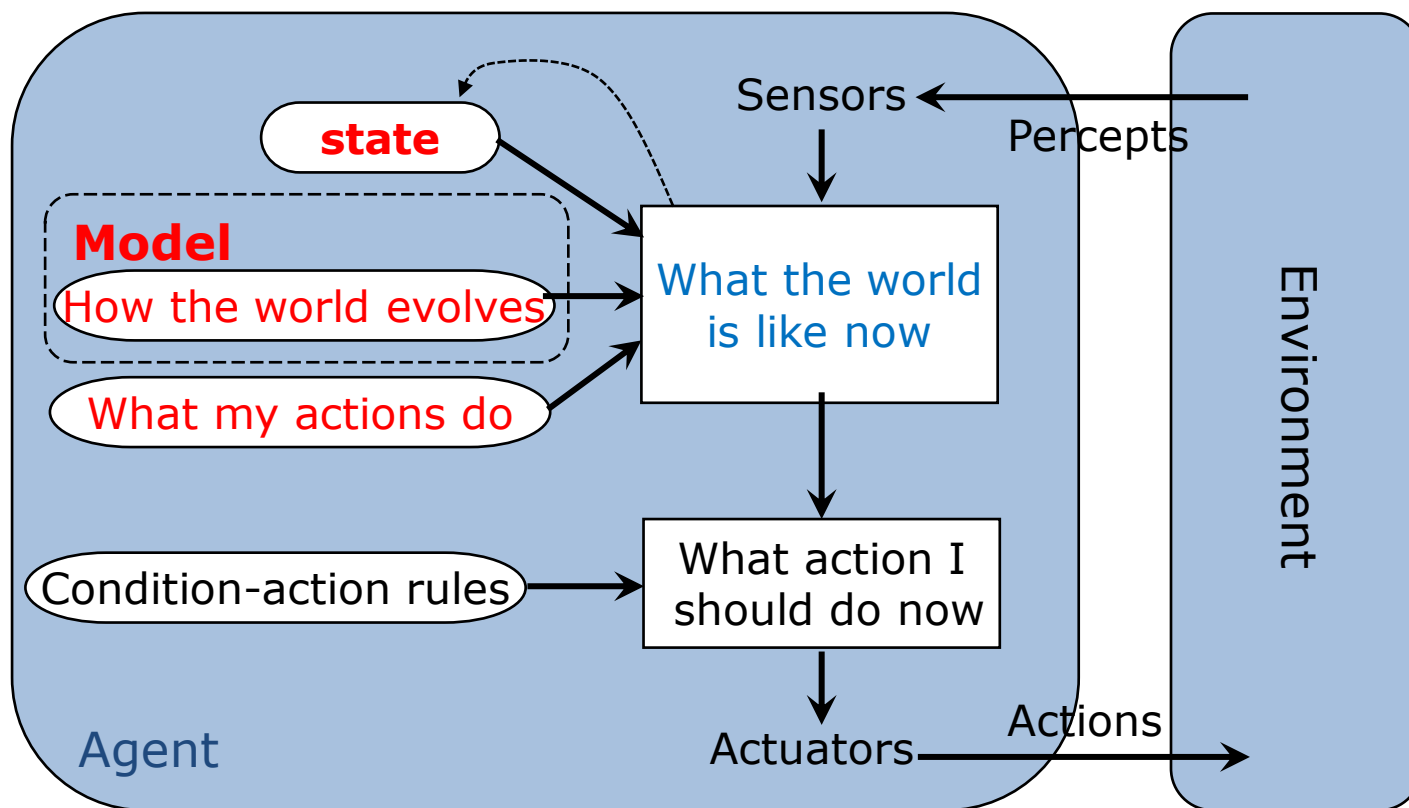
基于模型的反射Agent Model-based Reflex Agent

- 世界模型
 - 只有位置传感器时：干净区域在8小时后会变脏（世界是如何独立于Agent发展的）
 - 只有灰尘传感器时：动作Left会使得位置为A（Agent自身的行动如何影响世界）
- 结合过去的状态、执行的动作和当前感知估计当前状态

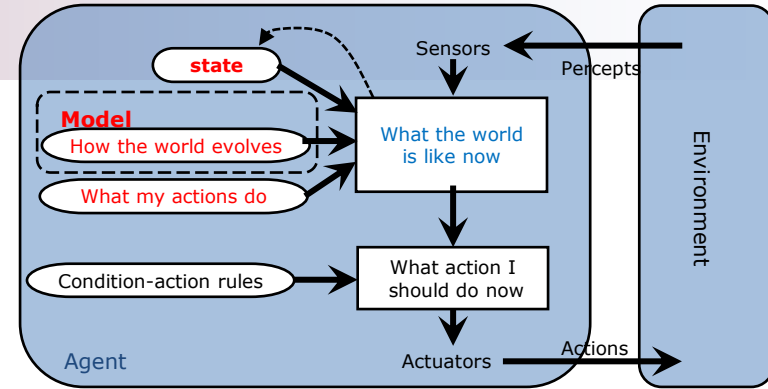


基于模型的反射Agent

- Maintain **internal state** to tackle **partially observable** environments.



基于模型的反射Agent



- Over time update state using world knowledge
 - How does the world change.
 - How do actions affect world. $\Rightarrow$ *Model of World*

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

static: *rules*, a set of condition-action rules

state, a description of the current world state

action, the most recent action

model, a description of how the next state depends on current state and action

state $\leftarrow$ UPDATE-STATE(*state*, *action*, *percept*, *model*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ rule.ACTION

return *action*

Agent自身的行动如何影响世界
世界是如何独立于Agent发展的



(3)基于目标的AGENT

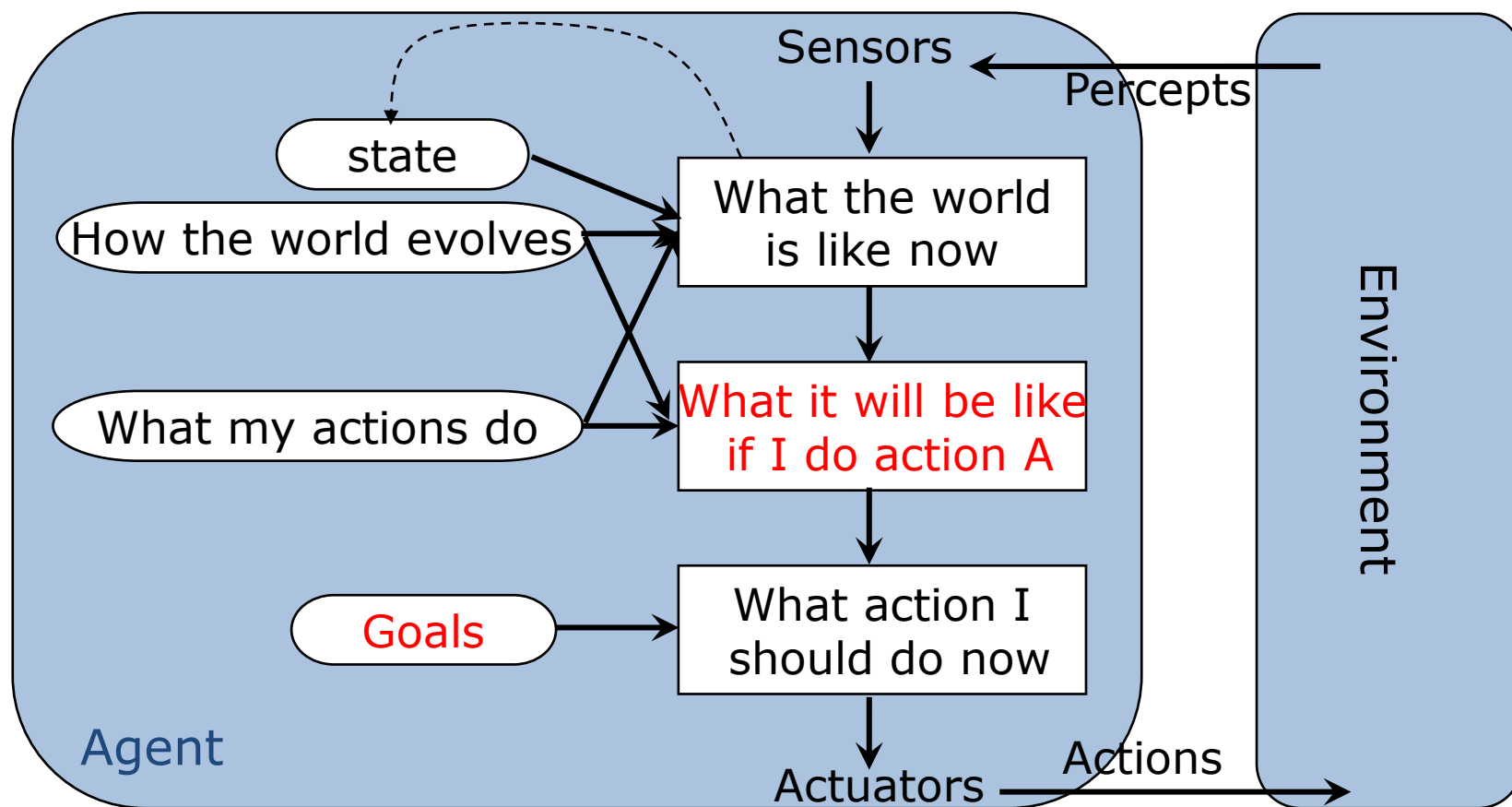
基于目标的Agent Goal-Based Agent

□知道当前的环境状态对决策而言并不够。Agent还需要目标信息来描述想要达到的状况——例如，要到达乘客的目的地。Agent程序可以把这种信息和模型相结合。



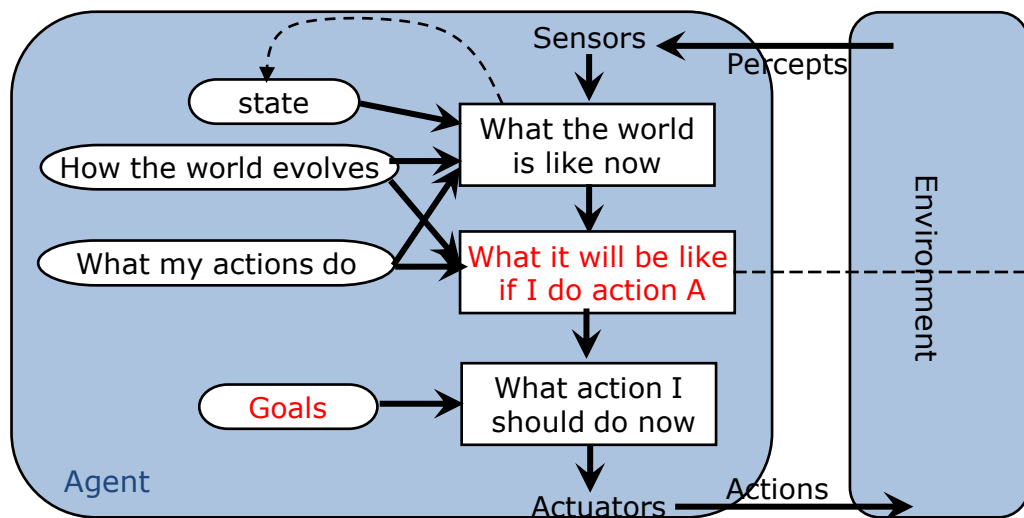
基于目标的Agent

□ 通常需要搜索和规划



基于目标的Agent会推理

- 反射Agent看到刹车灯时就刹车。
- 基于目标的Agent会推理，如果前车的刹车灯亮起，则它将要刹车减速：由于已知世界通常发展的方式，能够达到**不撞上其它车辆**的**目标**的唯一行动就是刹车。



预测执行动作 $a_1, a_2, \dots$ 后结果将是什么

简单反射Agent只能实现固定目标 基于目标的Agent可以修改目标

- 基于目标的Agent的行为在要前往不同的地点时改变起来很容易。
- 反射Agent关于何时转弯和何时直行的规则仅对于单一目的地是可行的；要前往新的目的地，很多规则都要更换。

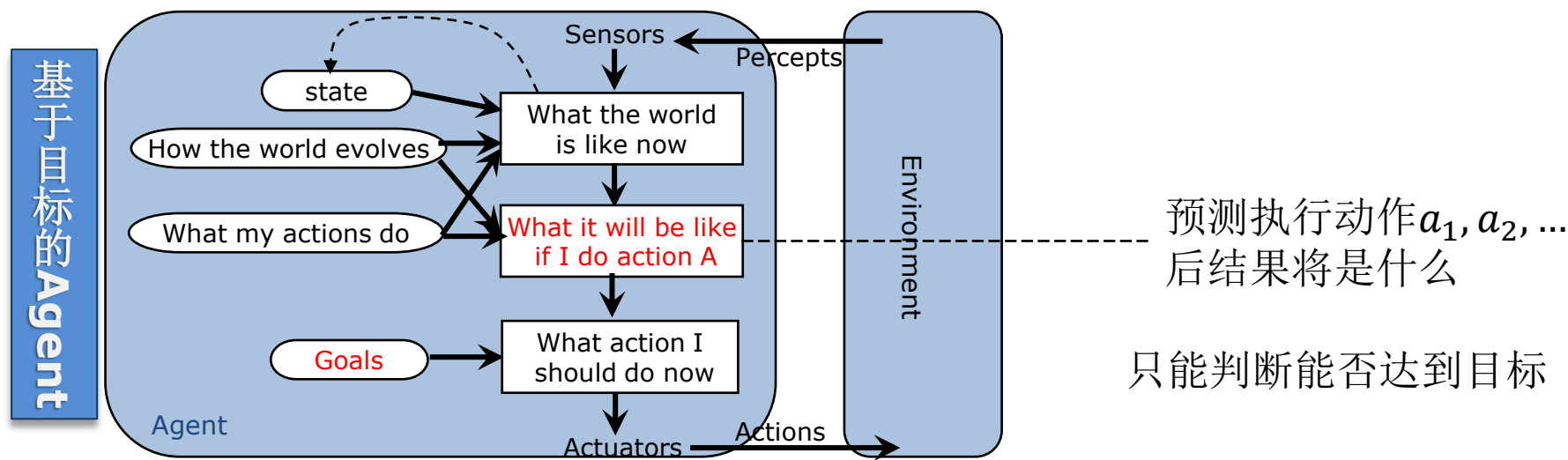


(4)基于效用的AGENT

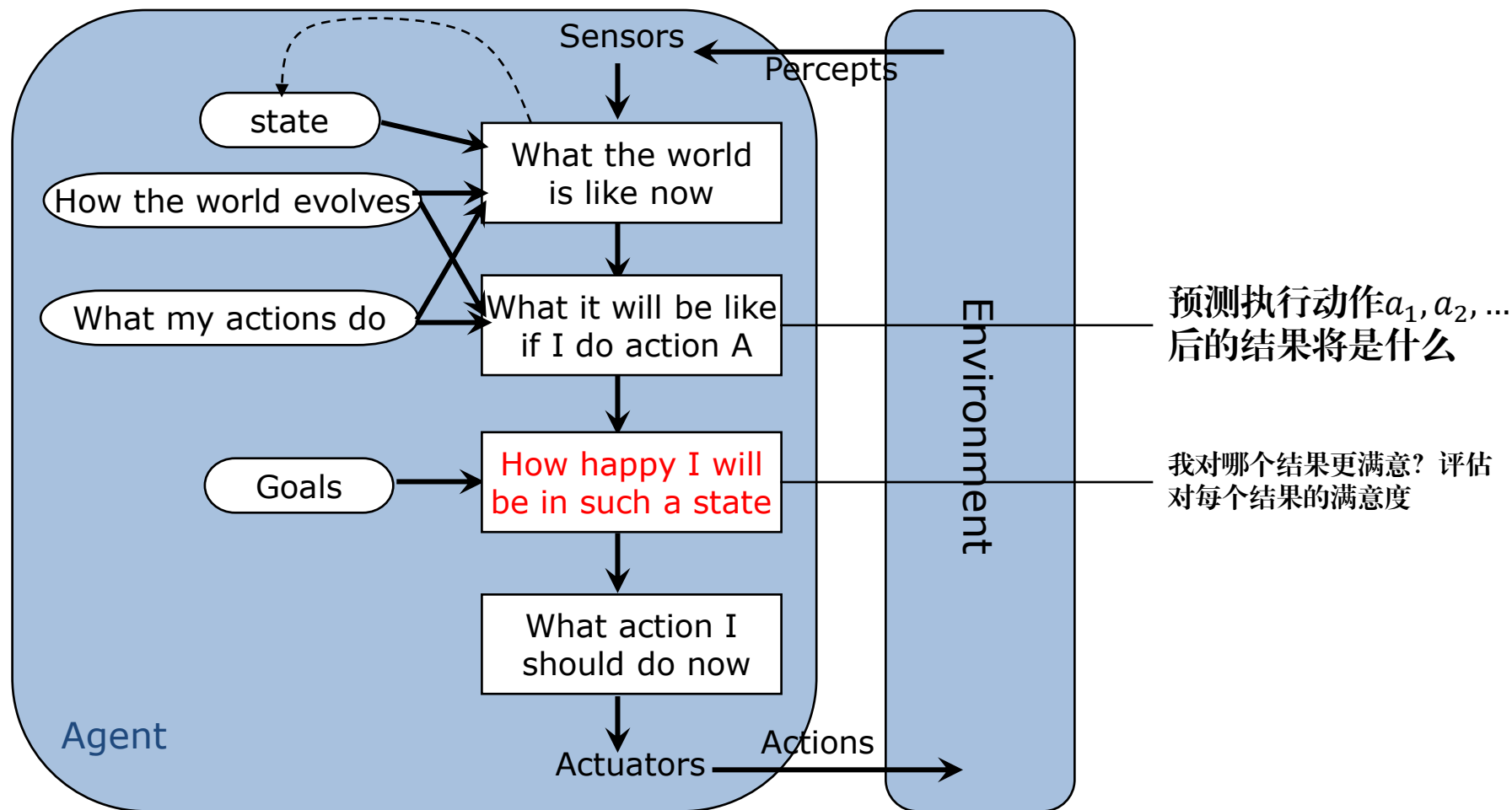
基于效用的Agent

Utility-Based Agent

- 仅靠目标可能不足以生成高品质的行为。例如，有很多行动序列可以让出租车到达目的地，但有些更快、更安全、更可靠，或者更便宜。**更通用的性能度量**应该允许不同的世界状态之间根据Agent的偏好（效用）进行比较。



基于效用的Agent



学习Agent的4个部件如何工作

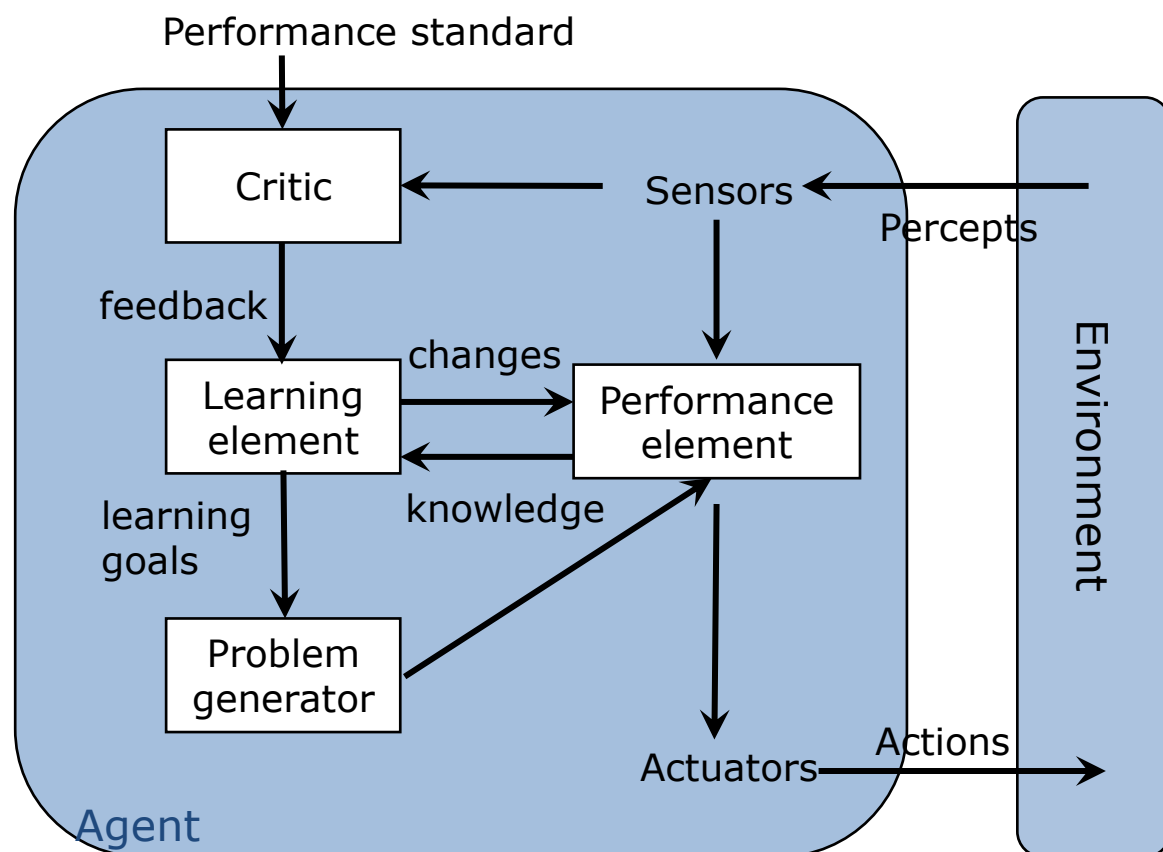
2 学习AGENT



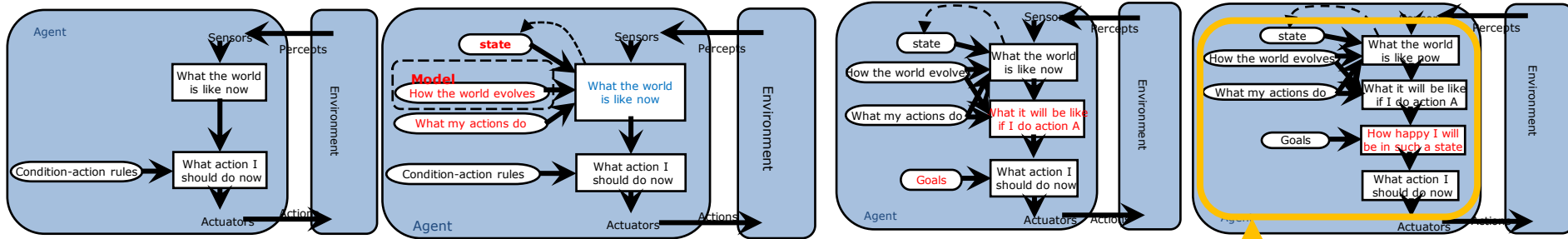
学习Agent-四个部件

□ 4 componets:

- *Performance element*
执行部件
- *Learning element*
学习部件
- *Critic*
评判部件
- *Problem generator*
问题生成器

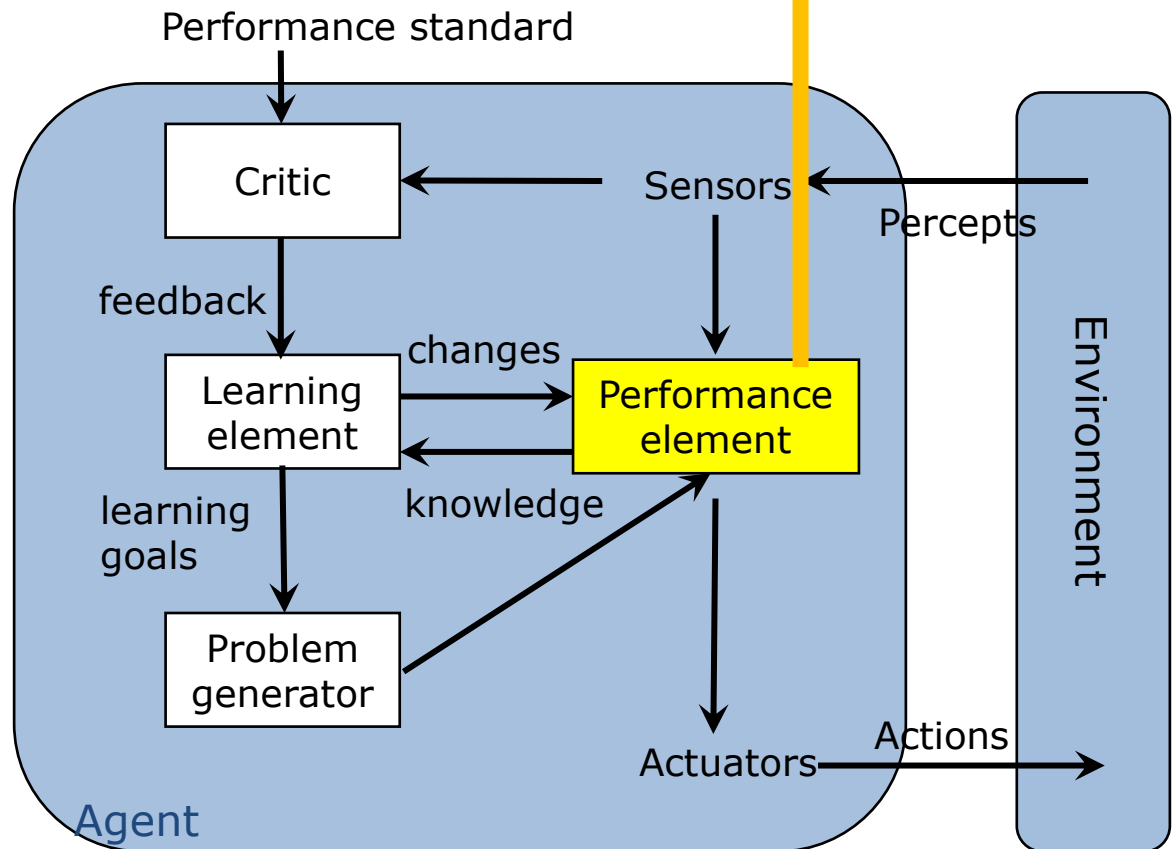


学习Agent-执行部件



□ *Performance element*: selecting actions based on percepts.

□ 执行部件就是前面考虑的整体Agent, 接受感知, 选择动作。

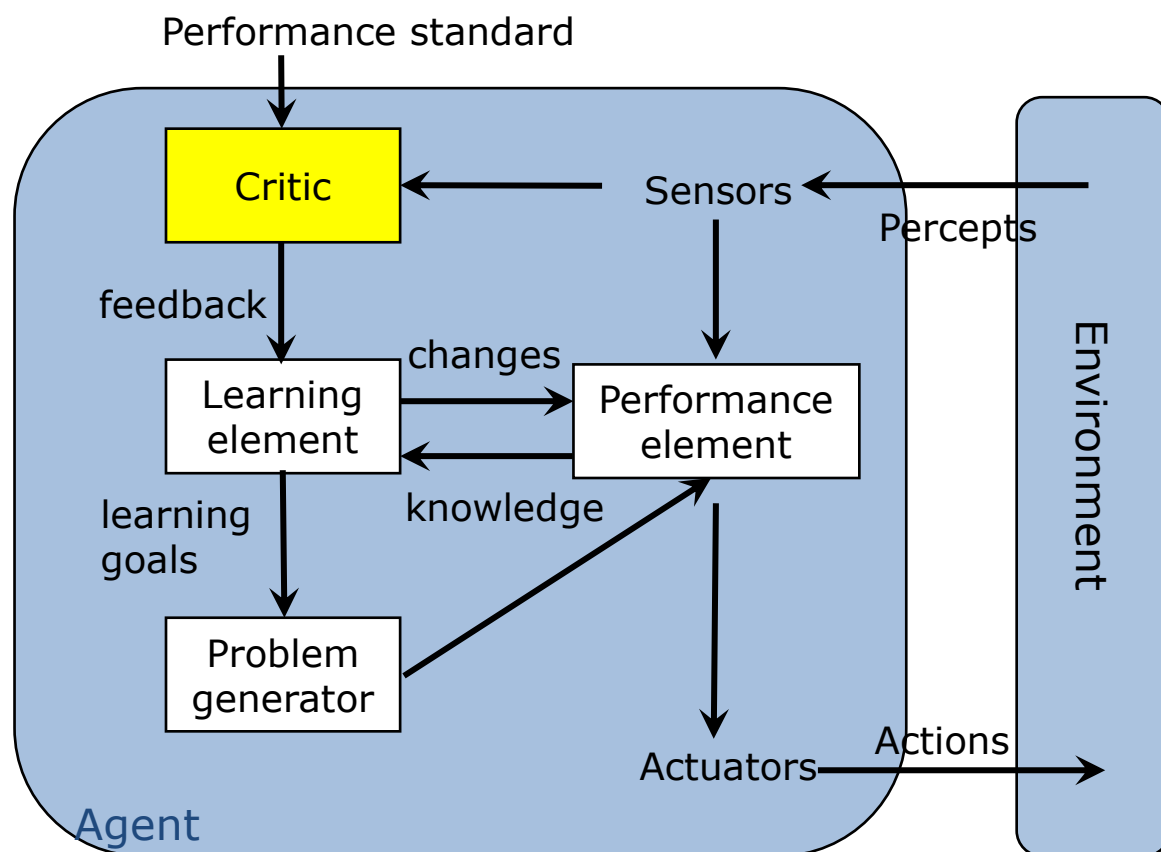


学习Agent-评判部件

□Critic

□评判部件根据性能标准告知学习部件Agent的运行情况。当“将军”发生了，评判部件告知学习部件：好事情发生了

□性能标准是固定的，Agent不应该修改性能标准来适应自己的行为



学习Agent-学习部件

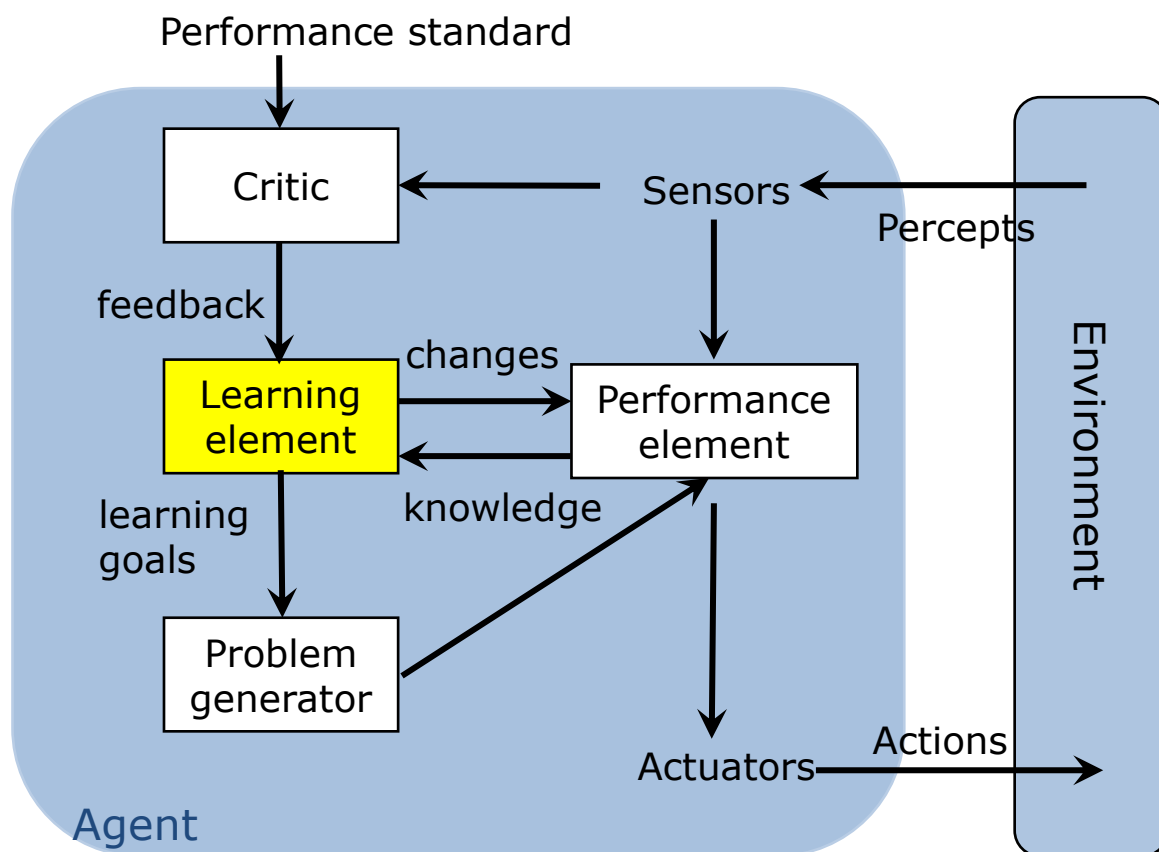
□ *Learning element*:

introduce

improvements in
performance element.

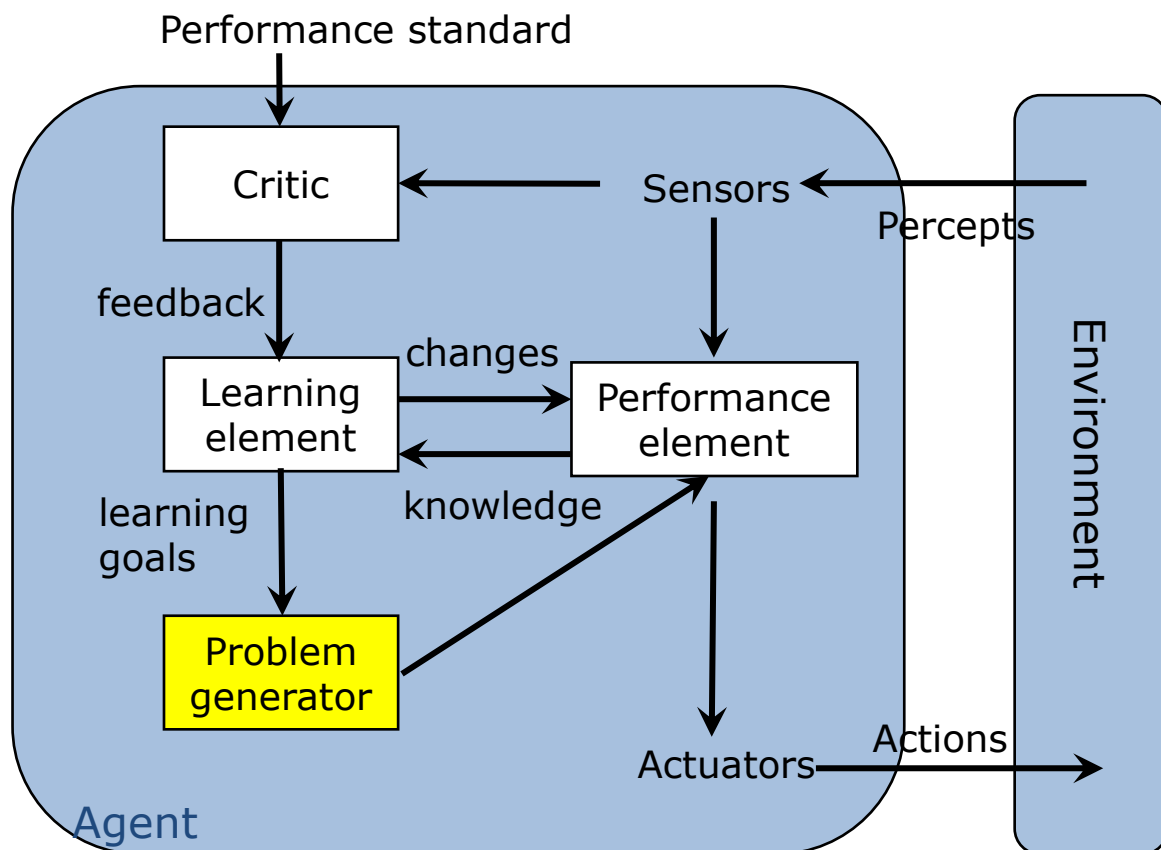
□ 学习部件根据评判部件的反馈评价Agent做得如何，从而确定如何修改执行部件。

$$action = f(percepts; \Theta)$$



学习Agent-问题生成器

- *Problem generator:* suggests actions that will lead to new and informative experiences.
- 如果执行部件自行其是，它会一直根据已知的知识采取最佳行动
- 问题生成器可向执行部件建议探索性行动，短期内可能是次优的行动，但长远而言可能是更好的行动



学习Agent

三个问题：

1. Agent需要什么样的技能？
2. Agent需要什么样的执行部件来行使这个技能？
3. 如何才能让Agent学到这个技能？

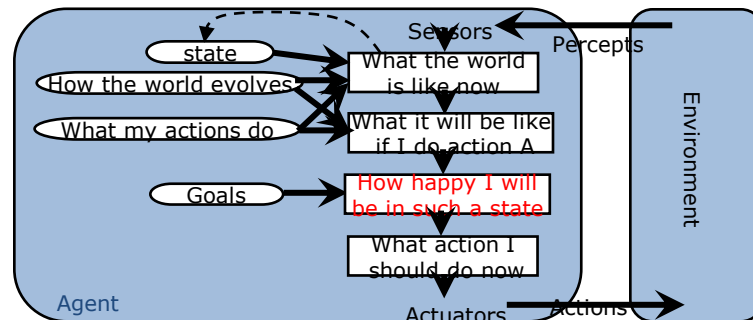
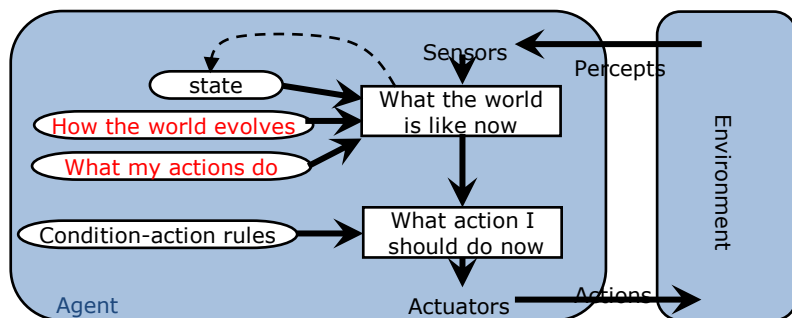
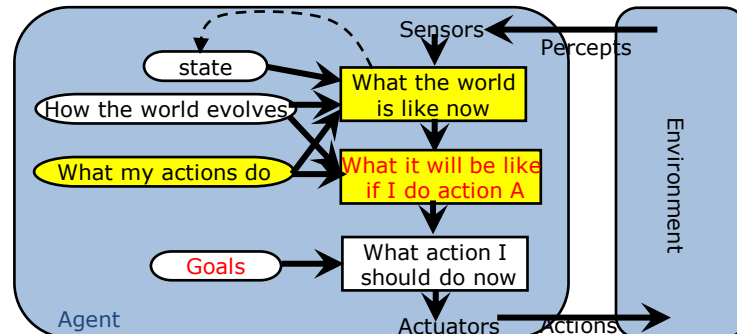
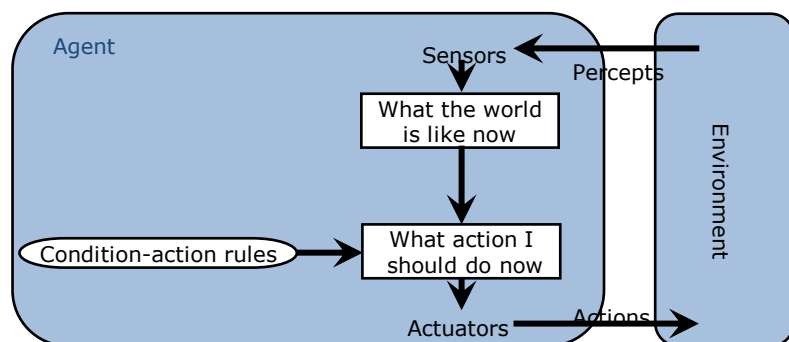
3种状态表示方法

3 AGENT的状态如何表示？



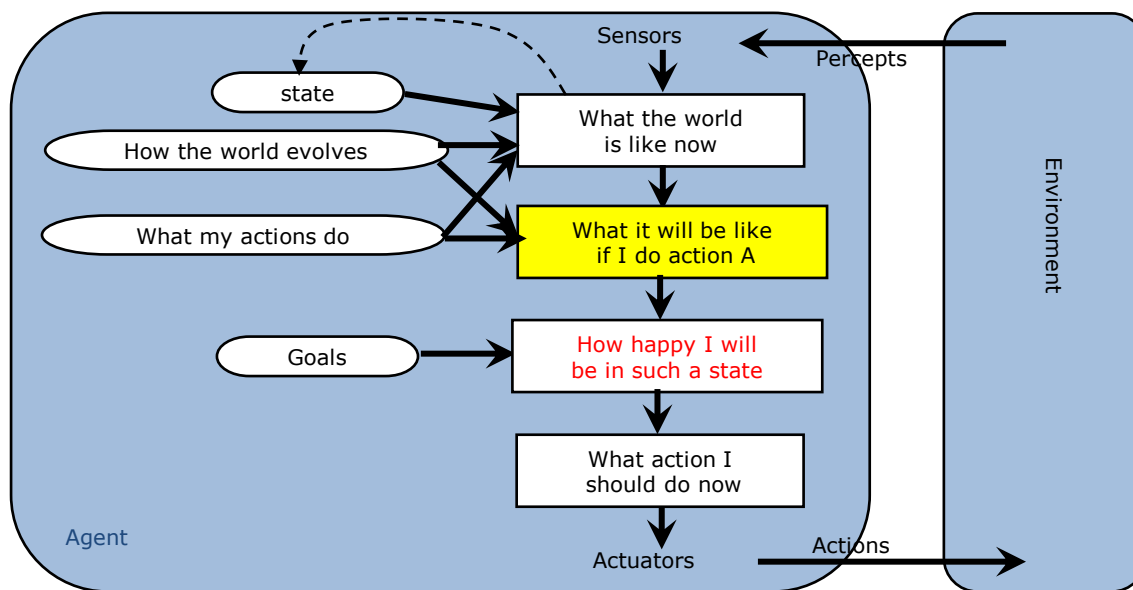
Agent的部件是如何工作的

- Agent程序包含回答以下问题的部件：“世界现在怎样了？” “我现在应该采取什么行动？” “**我的行动结果怎么样**”



Agent的部件是如何工作的

- 处理“**我的行动后果如何**”的部件描述了发生在环境中的变化，将之视为行动的后果。

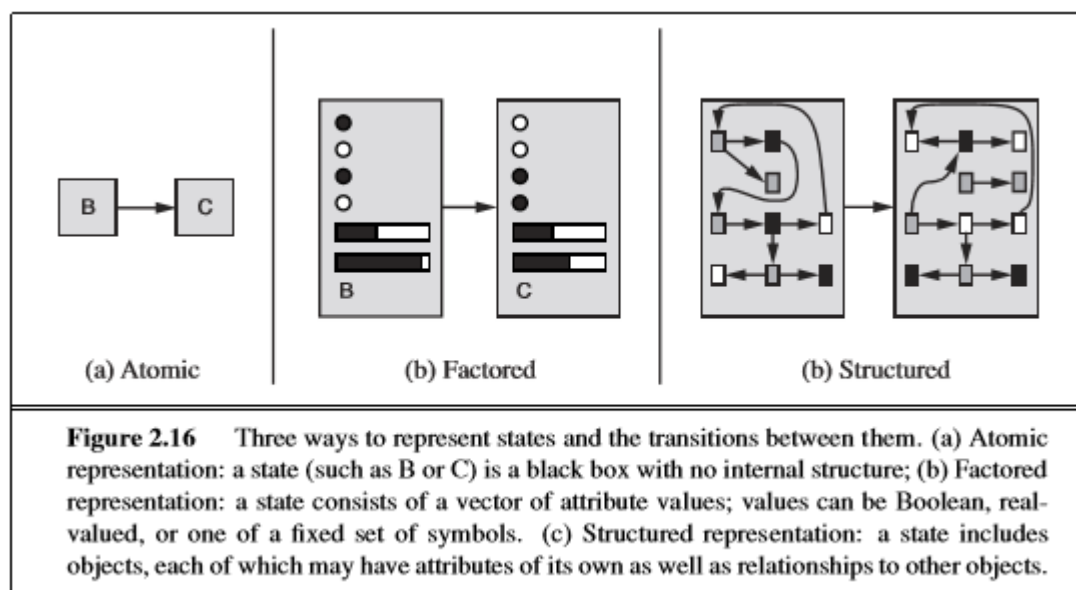


状态表示

- 处理“我的行动后果如何”的部件描述了发生在环境中的变化，将之视为行动的后果。
- 如何表示状态及其转换？
- 沿着复杂度和表达能力增长的轴线有三种表示：
原子表示、要素化表示、结构化表示
- 搜索和博弈论；命题逻辑、机器学习；一阶逻辑、自然语言理解分别采用什么表示？

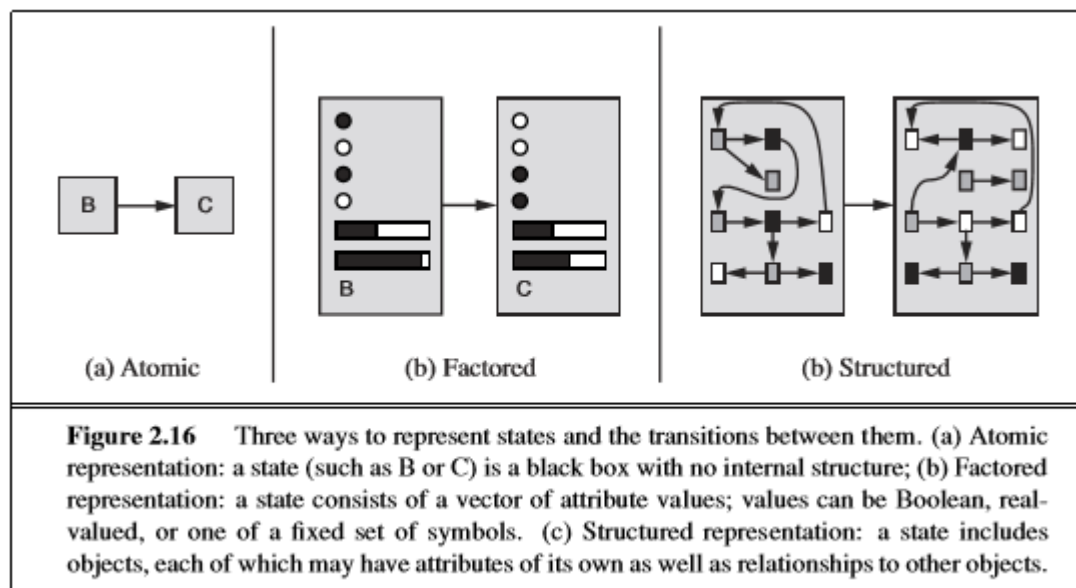
(1)原子表示

- 原子表示：罗马尼亚问题中，状态是城市的名字——原子——没有内部结构。
- 使用原子表示：搜索和博弈（ch03-05）、隐马尔科夫模型(ch15)、马尔科夫决策过程(ch17)



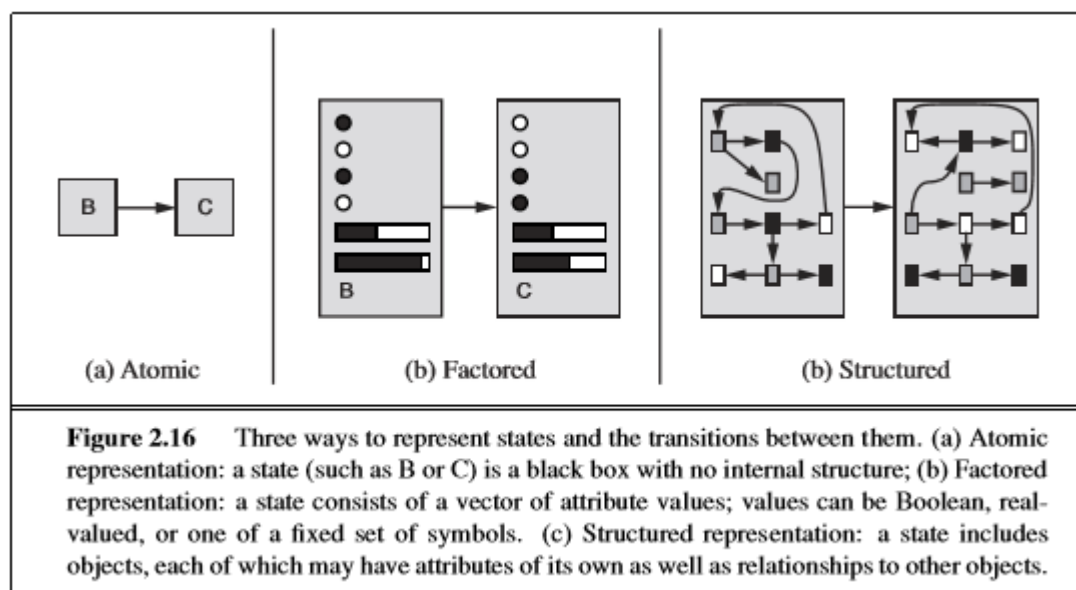
(2)要素化表示

- 要素化表示：状态表示为变量或特征的集合.例如(城市名,邮箱里还有多少油,卡上还有多少余额,目前的天气情况)
- 使用要素化表示：约束满足(ch06)、命题逻辑(ch07)、规划(ch10ch11)、贝叶斯网(ch13-16)、机器学习(ch18-21)



(3)结构化表示

- 结构化表示：世界中有事物，事物间有关联
- 使用结构化表示：关系数据库、一阶逻辑(ch8/9/12)、一阶概率模型(ch14)、基于知识的学习(ch19)、自然语言理解(ch22/23)



同时在所有点上操作

- 表达能力越强，表示就越简洁。
- 表达能力越强，推理和学习就越复杂。
- 为了找到更具表达力的表示，避免它们的缺点，现实世界的智能系统可能需要同时在轴线上的所有点上操作

- Agent程序的四种基本结构：
 - 简单反射Agent、
 - 基于模型的反射Agent、
 - 基于目标的Agent、
 - 基于效用的Agent
- 如何获得选择动作的能力：学习Agent
- 状态的表示：原子表示、要素化表示、结构化表示

Thanks

LU9UK2